UNITÀ 3 · APPLICAZIONI JAVA PER LA COMUNICAZIONE IN RETE

WebSocket: la chat nel browser

HTTP sa solo rispondere. Con una richiesta speciale la connessione cambia protocollo e diventa un canale sempre aperto, in cui anche il server può parlare per primo.

PER COMINCIARE

Quale app "in tempo reale" usi di più?

SONDAGGIO

- **A** Un gioco multiplayer
- **B** Un documento condiviso con i compagni
- **C** Una chat o Discord
- **D** Risultati sportivi in diretta
- **E** Un chatbot che scrive la risposta poco alla volta

IL PROBLEMA

In HTTP parla sempre prima il browser

Tecnica	Come funziona	Direzione	Quando usarla
Polling	Il browser chiede "novità?" ogni N secondi	browser → server	Aggiornamenti rari, semplicità
Long polling	Il server tiene la richiesta aperta finché ha una novità	browser → server	Vecchi sistemi, compatibilità
Server-Sent Events	Una risposta HTTP che non finisce mai: `text/event-stream`	solo server → browser	Notifiche, streaming di testo
WebSocket	La connessione TCP cambia protocollo e resta aperta	nei due sensi	Chat, giochi, collaborazione

METTITI ALLA PROVA

HTTP/1.1 classico: il server ha un messaggio nuovo per te. Come te lo manda?

QUIZ

- **A** Invia una risposta HTTP anche senza richiesta
- **B** Lo scrive in un cookie
- **C** Non può: deve aspettare che il browser chieda
- **D** Con un redirect 302 verso la pagina nuova

RFC 6455 · L'HANDSHAKE

Una richiesta HTTP che chiede di cambiare protocollo

```
browser → server
GET /chat/ws HTTP/1.1
Host: localhost:8080
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key:
  dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: http://localhost:8080
```

```
server → browser
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept:
  s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

da qui in poi: frame WebSocket
sulla stessa connessione TCP
```

Il server dimostra di aver capito la richiesta: **Accept = Base64(SHA-1(Key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))**

AL COMPUTER · 15 minuti

Calcola Sec-WebSocket-Accept

Crea **AcceptKey.java** nella cartella **u3-websocket**:

```
static String accept(String key) throws Exception {
    String guid = "258EAFA5-E914-47DA-95CA-C5AB0DC85B11";
    byte[] sha1 = MessageDigest.getInstance("SHA-1")
        .digest((key + guid).getBytes(UTF_8));
    return Base64.getEncoder().encodeToString(sha1);
}
```

Nel `main` stampa `accept("dGhlIHNhbXBsZSBub25jZQ==")`, poi prova una chiave a tua scelta.

Controllo: deve uscire esattamente `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`, la risposta dell'esempio della RFC 6455. Un solo carattere diverso nella stringa fissa cambia tutto il risultato.

DOPO IL 101

Niente più header: piccoli frame nei due sensi

2-14 byte di intestazione

FIN, un **opcode** (che tipo di frame è), la lunghezza e, dal browser, una maschera di 4 byte.

Testo o binario

Un messaggio può essere testo UTF-8 (per esempio JSON) o dati binari (immagini, audio, stato di un gioco).

Controllo

Close per chiudere con un codice, **ping e pong** per controllare che l'altro capo sia ancora vivo.

`ws://` usa la porta 80 ed è in chiaro; `wss://` usa la 443 dentro **TLS**. Fuori dal laboratorio si usa sempre `wss://`.

PUZZLE

Abbina ogni opcode al suo frame

0x1	---	Ping: "ci sei ancora?"
0x2	---	Frame binario
0x8	---	Chiusura, con un codice come 1000
0x9	---	Pong: la risposta al ping
0xA	---	Frame di testo UTF-8

IL CODICE · BROWSER

L'API WebSocket in JavaScript

```
// stesso host e cartella della pagina: ws://localhost:8080/chat/ws
const base = location.pathname.replace(/[^/]*$/, "");
const ws = new WebSocket((location.protocol === "https:" ? "wss://" : "ws://") + location.host + base + "ws");

ws.onopen = () => stato.textContent = "Connesso";
ws.onclose = (e) => stato.textContent = "Disconnesso (codice " + e.code + ")";
ws.onmessage = (e) => {
  const li = document.createElement("li");
  li.textContent = e.data;                    // mai innerHTML: niente XSS
  lista.append(li);
};
modulo.onsubmit = (ev) => {
  ev.preventDefault();
  ws.send(nick.value + ": " + testo.value);
  testo.value = "";
};
```

IL CODICE · SERVER

Un endpoint `jakarta.websocket` su Tomcat

```
@ServerEndpoint("/ws")
public class ChatEndpoint {
    private static final Set<Session> sessioni = ConcurrentHashMap.newKeySet();

    @OnOpen  public void apri(Session s)   { sessioni.add(s); }
    @OnClose public void chiudi(Session s) { sessioni.remove(s); }
    @OnError public void errore(Session s, Throwable t) { sessioni.remove(s); }

    @OnMessage
    public void messaggio(String testo, Session da) {
        for (Session s : sessioni) {
            if (!s.isOpen()) continue;
            try {
                synchronized (s) { s.getBasicRemote().sendText(testo); }   // un invio alla volta
            } catch (IOException e) { sessioni.remove(s); }
        }
    }
}
```

METTERE IN LINEA

Dove vanno i file in Tomcat 10.1

```
tomcat/
├─ bin/startup.bat   (.sh su Linux)
├─ lib/              websocket-api.jar…
└─ webapps/
   └─ chat/
      ├─ index.html
      └─ WEB-INF/
         └─ classes/
            └─ ChatEndpoint.class
```

1. Compila contro le librerie di Tomcat (un solo comando, su una riga):

```
javac -cp "tomcat/lib/*"
  -d tomcat/webapps/chat/WEB-INF/classes
  ChatEndpoint.java
```

2. Avvia (o riavvia) Tomcat con `startup`.
3. Apri `localhost:8080/chat/`: la pagina si collega a `ws://…/chat/ws`.

> Non serve `web.xml`: all'avvio Tomcat trova da solo le classi annotate con `@ServerEndpoint`. Attenzione alla versione: Tomcat 10.1 usa i pacchetti `jakarta.*`, Tomcat 9 i vecchi `javax.*`.

AL COMPUTER · 40 minuti

La chat WebSocket

1. Scompatta Tomcat 10.1 e crea la cartella **webapps/chat/WEB-INF/classes**.
2. Scrivi **index.html** in **webapps/chat** e **ChatEndpoint.java** nella tua cartella di lavoro.
3. Compila con il comando della slide precedente e avvia Tomcat.
4. Apri `http://localhost:8080/chat/` in **due schede** e scrivi da entrambe.
5. Fai uno screenshot con le due schede affiancate: **chat-ws.png**.

Controllo: lo stato diventa "Connesso" e ogni messaggio compare in tutte e due le schede, anche in quella che l'ha inviato. Se fermi Tomcat, la pagina mostra "Disconnesso" con un codice (di solito 1001 o 1006).

QUIZ A TRABOCCHETTO

I frame dal browser al server sono "mascherati" con 4 byte casuali. Perché?

QUIZ

- **A** Per cifrare i messaggi
- **B** Perché proxy e cache lungo la strada non li scambino per richieste HTTP
- **C** Per comprimerli
- **D** Per dimostrare l'identità dell'utente

AL COMPUTER · 20 minuti

Guarda dentro la connessione

1. Con la chat aperta premi **F12**, scheda **Rete** (Network), filtro **WS**, e ricarica la pagina.
2. Clicca sulla richiesta `ws`: annota lo stato e gli header `Upgrade`, `Sec-WebSocket-Key` e `Sec-WebSocket-Accept`.
3. Verifica la coppia chiave–risposta con il tuo **AcceptKey.java**.
4. Nella scheda **Messaggi** guarda i frame; poi invia `<b>ciao</b>`.
5. Scrivi tutto in **ispezione.txt**.

Controllo: lo stato è `101 Switching Protocols`; AcceptKey applicato alla tua chiave dà esattamente l'Accept del server. Il messaggio `<b>ciao</b>` compare con i simboli, non in grassetto.

MENTI CHE CI HANNO PRECEDUTO

Il Web non era nato per il tempo reale

> **Il modo migliore per prevedere il futuro è inventarlo.**
>
> **Alan Kay**, informatico, a una riunione allo Xerox PARC nel 1971

2008 → 2011

Il Web era fatto per documenti. Gli sviluppatori inventarono trucchi come il long polling; nel 2008 Ian Hickson e Michael Carter proposero il nome **WebSocket**; nel 2011 l'IETF pubblicò la **RFC 6455**.

IL TEMPO REALE OGGI

Dalla chat all'intelligenza artificiale





PUZZLE

Metti in ordine la vita di una connessione WebSocket

1. Uno dei due manda un frame di chiusura con il codice 1000
2. Parte una GET con `Upgrade: websocket` e la chiave
3. Il browser scarica la pagina con una normale richiesta HTTP
4. Lo script esegue `new WebSocket(url)`
5. Client e server si scambiano frame nei due sensi
6. Il server risponde `101` con `Sec-WebSocket-Accept`

ALL'ESAME

Scegliere e giustificare la tecnologia

Requisito del progetto	Scelta	Perché
Bacheca che si aggiorna ogni ora	Polling (o ricarica)	Rari aggiornamenti: la soluzione più semplice basta
Notifiche e stato di un ordine	Server-Sent Events	Solo dal server al client, HTTP normale, riconnessione automatica
Chat, gioco, lavagna condivisa	WebSocket	Messaggi frequenti nei due sensi, overhead minimo
Videochiamata	WebRTC	Audio e video diretti tra i browser, su UDP

> In una prova di progetto la risposta "uso WebSocket" vale poco da sola: vale il **perché**, confrontato con almeno un'alternativa.

BIGLIETTO D'USCITA

Completa prima di uscire

Il server accetta il cambio di protocollo con il codice di stato _____.

L'header che chiede il cambio di protocollo si chiama _____.

WebSocket cifrato con TLS usa lo schema _____.

Nel browser il messaggio ricevuto arriva nell'evento _____.

Su Tomcat la classe dell'endpoint si annota con _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- HTTP: parla prima il browser. Polling, SSE e WebSocket aggirano il limite
- Handshake: GET con Upgrade, risposta 101, Accept da SHA-1 e Base64
- Poi frame piccoli: testo, binario, close, ping e pong
- Browser: `new WebSocket`, `onmessage`, `send`, sempre textContent
- Tomcat: @ServerEndpoint, @OnOpen, @OnMessage, @OnClose
- Maschera ≠ cifratura: fuori dal lab sempre wss://

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.